

Course week 1

Algorithm

Algorithm

Question 1 1/1

Algorithn is

- ☐ Another name for a function.
- ☐ Description of a computer program.
- ☒ Description of how to solve a problem.

Question 2 1/1

Algorithms should also be

- ☒ Unambiguous
- ☐ Straight-forward to program
- ☐ Simple

Pseudocode

Pseudocode

Question 1 1/1

Pseudocode is

- ☐ Standardized.
- ☒ Meant only to be read by humans.
- ☐ Ready to be converted word by word, for example, into python-code.

Question 2 1/1

The purpose of pseudocode is

- ☐ To be a ready made code as is.
- ☒ Present an algorithmic operational principle.
- ☐ Be an implementation example of the algorithm.

Decrease and conquer

Decrease and conquer

Question 1 1/1

Using decrease and conquer - design strategy

- ☐ ... one divides the problem into smaller pieces, which will be handled one at a time.
- ☒ ... one takes a suitable part of the programs, which will be solved. This will be repeated, until the whole problem is solved.
- ☐ ... the problem is handled as one big piece.

Insertion sort

Insertion sort

Question 1 1/1

Insertion sort sorts items

- ☒ By looking for a placement for each item, one by one, by moving already ordered items.
- ☐ By comparing random items and switching their locations if needed.
- ☐ By comparing each adjacent item and switching their locations if needed.

Question 2 1/1

With large datasets insertion sort is

- ☐ Is as efficient as with smaller datasets.
- ☒ Becomes very inefficient.
- ☐ Becomes very efficient.

From pseudocode to implementation

From pseudocode to implementation

Question 1 2/2

When one start to implement an algorithm, it is important to

- ☐ Follow strictly the implementation presented by pseudocode.
- ☒ Perform input validation and take care of error handling.
- ☒ Think what would be the most reasonable way to implement it using the programming language at hand.
- ☐ To write as many lines of code as in the the pseudocode.

Algorithm efficiency

Algorithm efficiency

Question 1 1/1

What do we actually measure when measuring an algorithm's efficiency?

- ☒ The number of executed operations.
- ☐ Milliseconds.
- ☐ The complexity of the pseudocode.
- ☐ The number of pseudocode rows.

Asympthotic efficiency

Algorithm efficiency

Question 1 1/1

Why should one study the asymptotic efficiency of an algorithm?

- ☐ It can be used to estimate the needed lines of code to implement the algorithm.
- ☒ Because it give at least some estimate of the actual efficiency if the algorithm.
- ☐ It allows you to calculate the real time needed to execute the implemented algorithm.

Question 2 2/2

Which statements are true?

- ☐ The asymptotic efficiency of an algorithm tells exact limits for its actual efficiency.
- ☒ The same algorithm can have several reasonable estimates of efficiency based on different initial values.
- ☒ Asymptotic efficiency describes the speed of slowing down of the algorithm, when the amount of data gets bigger.
- ☐ An efficient algorithm always slows down very fast.

Sharing data among data structures

Sharing data among data structures

Question 1 3/3

Sharing the data wisely helps (you can pick several)

- ☒ In the upkeep of the data 有助于数据的维护/管理
- ☐ With the readability of the code
- ☒ To prevent unnecessary waste of storage
- ☒ To speed up the program 因为不用复制很多重复数据，程序处理数据时可能更快
- ☐ To write the code

Recursion

Recursion

Question 1 1/1

Recursion is especially useful in this course, because

- ☐ It is just another way to implement a loop.
- ☐ Really complicated problems can be solved easily using it.
- ☒ It is easy to solve big problems, which contain similar smaller problems.
- ☐ It differs notably from the normal human way of solving problems.

Exercise about reading pseudocode

Exercise about reading pseudocode

Question 1 2/2

Study the following pseudocode:

```
1 Mystery1(Array[a1, a2, ... , aN]):  
2   B := []  
3   FOR elem IN Array LOOP  
4     IF first THEN  
5       B[0] := elem  
6     ELSEIF elem > LAST(B)  
7       APPEND(B, elem)  
8     END IF  
9   END LOOP  
10  RETURN B
```

What should a program described by the pseudocode return for [3, 2, 5, 5, 8, 17, 16] as input?

- ☐ [3, 2, 16]
- ☐ [2, 5, 16]
- ☒ [3, 5, 8, 17]
- ☐ [3, 2, 5, 5, 8, 17, 16]

Question 2 2/2

Study the following pseudocode:

```
1 Mystery2(Int A, Int B): # A ja B ovat aina > 0  
2   res := 0  
3   WHILE B > 0 LOOP  
4     IF B % 2 == 1 THEN  
5       res := res + A  
6     ENDIF  
7     A = A * 2  
8     B = B / 2  
9   END LOOP  
10  IF res == 0 THEN  
11    RETURN A  
12  ELSE  
13    RETURN res  
14  ENDIF
```

What should a program described by the pseudocode return for 4 and 3 as input? Division will be rounded down to nearest integer.

- ☐ 0
- ☐ 1
- ☐ 4
- ☐ 7
- ☒ 12
- ☐ 64

Question 3 2/2

What does the previous algorithm describe?

- ☐ Addition
- ☒ Multiplication
- ☐ Subtraction
- ☐ Exponentiation
- ☐ None of the above

Recursive function

Recursive function

What should be written in place of the question marks, so that the recursive functions would work?

Question 1 5/5

```
int sumToK(int k) {  
    if (k <= 0) {  
        return 0;  
    } else {  
        return <<???>>;  
    }  
}
```

SumToK-function should add up all integers in between [1, k]. Example call: `sumToK(5) -> 15`

k + sumToK(k - 1)

✓ Correct!

Question 2 2/2

```
int sumOfDigits(int number) {  
    if (number < 10){  
        return number;  
    }  
    return <<???>>;  
}
```

Example call: `sumOfDigits(1234) -> 10` (Hint: modulus-operator %)

sumOfDigits(number / 10) + number % 10

✓ Correct!

Efficacy of the functions

Efficacy

Study the pseudocode functions below, and recall how insertion sort work. You can assume that only the aspects in the pseudocode will affect the efficacy significantly, and for example the efficacy of APPEND()-function doesn't need to be considered.

```
1 FunctionA(Array A):  
2   res := [[]] // empty 2D Array  
3   FOR i IN SIZE(A):  
4     FOR j IN SIZE(A):  
5       APPEND(res[i], A[i] * A[j])  
6     END LOOP  
7   APPEND(res, []) // Add empty array inside the 2D array  
8   END LOOP  
9   RETURN res
```

```
1 FunctionB(Array A, Int target)  
2   FOR i IN A:  
3     FOR j IN A:  
4       IF i * j == target THEN  
5         RETURN [i, j]  
6       END IF  
7     END LOOP  
8   END LOOP
```

```
1 FunctionC(Array A):  
2   IF SIZE(A) == 1 OR EMPTY(A) THEN  
3     RETURN TRUE  
4   ELSE IF FIRST(A) != LAST(A) THEN  
5     RETURN FALSE  
6   END IF  
7   POP_FIRST_AND_LAST(A) // Remove first and last elements  
8   FunctionC(A)
```

Efficacy of the functions

Question 1 2/2

Which functions do not necessarily slow down at all, when the amount of data increases?

- ☐ FunctionA
- ☒ FunctionB
- ☒ FunctionC
- ☐ InsertionSort

Question 2 2/2

In what case FunctionB is the most efficient, what is its best case?

- ☐ All items have the same value
- ☐ All items are in reverse order
- ☒ First item fulfills the search criteria

Question 3 2/2

How about the worst case for the same function?

- ☐ All items have different value
- ☐ All items fulfill the search criteria
- ☒ None of the pairs fulfill the criteria
- ☐ The items are in random order

Question 4 2/2

For which function the best, the worst and the median efficacy are the same?

- ☒ FunctionA
- ☐ FunctionB
- ☐ FunctionC
- ☐ InsertionSort

Insertion Sort

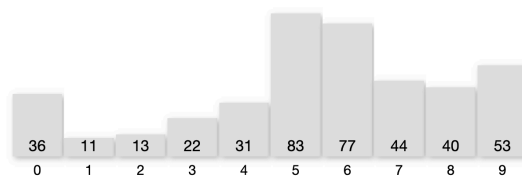
<https://plus.tuni.fi/COMP.CS.300/spring-2026/week1/content/?hl=en#chapter-exercise-1>

Ohjeet: Use Insertion Sort to sort the table given below in ascending order. Click on an item to select it and copy its value to another position by clicking on the new position.

```
1. def insortshift(A):
2.     for i in range(1, len(A)): # Insert i'th record
3.         temp = A[i]
4.         j=i
5.         while (j > 0) and (temp < A[j-1]):
6.             A[j] = A[j-1]
7.             j -= 1
8.             A[j] = temp
```

Undo Reset Model Answer Grade

Table to be sorted



temp variable

17

Course week 2

Divide and conquer - design principle

Divide and conquer - design principle

Question 1 1/1

"In divide and conquer - design principle, the problem is...

- ☒ divided into sub-problems, until it is trivial to solve, after which the partial solutions are collected as the final solution.
- ☐ divided into equal size sub-problems, after which each sub-problem is solved one at the time.
- ☐ converted into a similar problem, that we already know a solution for, and by changing the solution to fit the original problem.

Question 2 2/2

What types of problems are suited well for "divide and conquer" - design principle?

- ☒ Recursive
- ☐ Small
- ☐ Difficult to define
- ☐ Trivial
- ☒ Hierarchical

Quicksort

Quicksort

Question 1 1/1

Quicksort starts by

- ☐ randomizing the order of the items.
- ☒ selection one item to be a "pivot"-item.
- ☐ searches the placement of the biggest item.

Question 2 1/1

after which the items are

- ☒ sorted (partitioned) into two parts: the smaller or the larger than pivot items.
- ☐ moved randomly either on the right side of the pivot, or on the left side of the pivot.
- ☐ moved on the left side of the biggest item.

Question 3 1/1

Then items on both sides of the pivot are

- ☐ Ordered using a better sorting algorithm.
- ☒ Sent as a parameter to a recursive call to quicksort.
- ☐ The partitioning function is called again.

Question 4 1/1

This will be repeated, until

- ☐ The pivot-item is the biggest item.
- ☐ The items will accidentally be in a correct order.
- ☒ There are so few items left that they are already in the right order.

Partitioning in quicksort

Partitioning in quicksort

Question 1 2/2

The idea behind partitioning is

- ☐ To select such a set of items that all items fulfil the given condition.
- ☐ To select several items that fulfil the given condition and return them.
- ☒ To divide the items into to sub-sets for which the items in the first sub-set fulfil the given condition.

Mergesort

Mergesort

Question 1 1/1

Mergesort performs sorting in two distinctive steps, first

- ☐ the items are divided into sections that are already in order.
- ☒ the items are divided recursively into sets, until each set has just one item.
- ☐ the items are divided directly into $N/2$ sub-sets.

Question 2 1/1

After this

- ☐ we search for the smallest sub-set, and combine it to the final list first. This will be repeated until no more sub-sets are left.
- ☐ the other sub-sets are merged into the first section, one by one.
- ☒ the already ordered sections are merged together recursively.

Mergesort (merging)

Mergesort (merging)

Question 1 1/1

Merging combines two already ordered arrays together by moving

- ☐ the first items in each array into the end of a bigger array and by repeating this until one of the arrays are empty.
- ☐ the first item from each array, by taking turns, in the end of a bigger array.
- ☒ the smallest of the first items of the arrays into a new array, and by repeating this until both arrays are empty.

Algorithm efficiency

Algorithm efficiency

Question 1 1/1

O-notation refers to

- ☐ The upperbound of the slowing down of the function, when the amount of data gets bigger and there is a small amount of data.
- ☐ The lowerbound of the slowing down of the function, when the amount of data gets bigger.
- ☒ The upperbound of the slowing down of the function, when the amount of data gets bigger and there is a big amount of data.

Question 2 2/2

Select the right order from fastest to slowest:

- ☐ $O(N)$ - $O(\log N)$ - $O(N^2)$ - $O(N \log N)$ - $O(1)$
- ☒ $O(1)$ - $O(\log N)$ - $O(N)$ - $O(N \log N)$ - $O(N^2)$
- ☐ $O(1)$ - $O(N)$ - $O(\log N)$ - $O(N^2)$ - $O(N \log N)$
- ☐ $O(\log N)$ - $O(N)$ - $O(N \log N)$ - $O(N^2)$ - $O(1)$

Ω (Omega) and Θ (Theta) notation

Ω (Omega) and Θ (Theta) notation

Question 1 1/1

Omega-notation refers to

- ☒ The lowerbound of slowing down of the algorithm, when the amount of data gets bigger, and there is a big amount of data.
- ☐ The lowerbound of slowing down of the algorithm, when the amount of data gets bigger, and there is a small amount of data.
- ☐ The upperbound of slowing down of the algorithm, when the amount of data gets bigger.

Question 2 1/1

Theta-notation refers to

- ☐ The average slowing down of the algorithm, when the amount of data gets bigger.
- ☒ The exact order of magnitude in which the efficiency of the algorithm is in all cases.
- ☐ The midpoint between O and Omega.

Asymptotic analysis

Asymptotic analysis

```
1 Find(Array arr, Int key):  
2   FOR elem IN arr:  
3     IF elem == key:  
4       RETURN true;  
5     ENDIF  
6   ENDLLOOP  
7   RETURN false;
```

Question 1 2/2

Which statements are true for Find-algorithm (see above)?

- ☒ $O(N)$
- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $\Omega(1)$
- ☐ $\Omega(\log N)$
- ☐ $\Omega(N)$

Sorting algorithm comparision

Sorting algorithm comparision

Question 1 3/3

After comparing the pseudocodes of the sorting algorithms, you would be able to say that...

- ☒ Mergesort is always as fast as always inspite of the data.
- ☒ Insertionsort is rather inefficient in average.
- ☐ Insertionsort is the most inefficient sorting algorithm.
- ☐ In the worst case, Quicksort is slower than Insertionsort.
- ☒ $N \log N$ is the most common order of magnitude for sorting algorithms.
- ☐ One of the covered sorting algorithms is clearly better than the others.
- ☐ Quicksort is better on average than Mergesort because it takes less memory.
- ☒ The order of efficiency of the sorting algorithms depends on the initial state of the items to be sorted.

Asymptotic notations comparision

Asymptotic notations comparision

Question 1 1/1

If algorithm has order of magnitude $O(N)$, it also has order of magnitude:

- ☐ $O(\log N)$
- ☐ $\Omega(N)$
- ☒ $O(N \log N)$

Question 2 1/1

If algorithm has order of magnitude $\Omega(\log N)$, it also has order of magnitude:

- ☒ $\Omega(1)$
- ☐ $\Omega(N)$
- ☐ $\Theta(\log N)$

Quicksort

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week2/content/?hl=en>

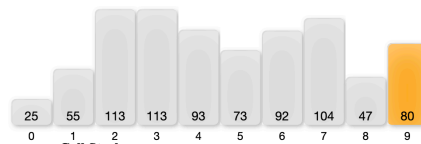
Ohjeet: In this exercise you are supposed to sort the array below by simulating the Quicksort algorithm. The pivot is colored orange, and picked and moved to the end of the sublist automatically. Your task in this exercise is to **perform the partitioning, move the pivot into place after the partitioning and to make the function calls**.

To select a bar, simply click on it. You can then click on another bar to swap it with the selected bar. The recursive Quicksort call can be made by clicking the **call** button and selecting the bounds for the function. Return from the function by clicking the **return** button. The call stack under the array shows the bounds for which the Quicksort function has been called.

```
1. int partition(Comparable[] A, int left, int right, Comparable pivot) {
2.     while (left <= right) { // Move bounds inward until they meet
3.         while (A[left].compareTo(pivot) < 0) left++;
4.         while ((right >= left) && (A[right].compareTo(pivot) >= 0)) right--;
5.         if (right > left) swap(A, left, right); // Swap out-of-place values
6.     }
7.     return left;           // Return first position in right partition
8. }
9.
10. void quicksort(Comparable[] A, int i, int j) { // Quicksort
11.     int pivotindex = findpivot(A, i, j); // Pick a pivot
12.     swap(A, pivotindex, j);             // Stick pivot at end
13.     // k will be the first position in the right subarray
14.     int k = partition(A, i, j-1, A[j]);
15.     swap(A, k, j);                      // Put pivot in place
16.     if ((k-i) > 1) quicksort(A, i, k-1); // Sort left partition
17.     if ((j-k) > 1) quicksort(A, k+1, j); // Sort right partition
18. }
```

Undo Reset Model Answer Grade

Array to be sorted



Call Stack

Left: 0, Right: 9

Call
Return

Mergesort

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week2/content/?hl=en>

UndoResetModel AnswerGrade

Ohjeet: Start at the bottom left. Merge two single element arrays to sort a sorted two-element array. Continue merging until you reach an array whose child arrays have not BOTH been sorted. Return to single element arrays and repeat the merging process as necessary until all elements have been merged into a single, sorted array. To merge an element into another array, click the element to select it, then click the position where it should be in the sorted, merged array. Remember, the order in which blocks are merged matters so be sure to select the smallest blocks first, starting at the left.

O-notation

O-notation

The vector v given as a parameter has size N . Select the strictest possible approximation.

Question 1 2/2

```
1 Mean(Vector v):
2   int sum = 0
3   FOR elem IN v :
4     sum = sum + elem
5   END LOOP
6   RETURN sum/SIZE(v)
```

What is the efficiency of the function Mean()?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $O(N)$
- ☐ $O(N \log N)$
- ☐ $O(N^2)$

Question 2 2/2

```
1 MeanOfRange(Vector v, int start, int end):
2   int sum = 0
3   FOR elem IN RANGE FROM start TO end :
4     sum = sum + elem
5   END LOOP
6   RETURN sum/(end-start +1)
```

What is the efficiency of the function MeanOfRange()?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $O(N)$
- ☐ $O(N \log N)$
- ☐ $O(N^2)$

Question 3 2/2

```
1 FindIfOutlier(Vector v, int max_var):
2   FOR elem IN v:
3     IF max_var < |elem - Mean(v)|:
4       RETURN elem
5   END
6   END
```

What is the efficiency of the function FindIfOutliers() when Mean() is taken from the previous exercise?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☐ $O(N \log N)$
- ☒ $O(N^2)$

Asymptotic efficiency

Asymptotic efficiency

```
1 Mystery1(Array[a1, a2, ... , aN]):
2   B := []
3   FOR elem IN Array LOOP
4     IF first THEN
5       B[0] := elem
6     ELSEIF elem > LAST(B)
7       APPEND(B, elem)
8     END IF
9   END LOOP
10  RETURN B
```

Question 1 2/2

What is the efficiency of the above algorithm when APPEND-function takes constant time (e.g. $\Theta(1)$)?

- ☒ Always linear, e.g. $O(N)$, $\Omega(N)$ and $\Theta(N)$.
- ☐ $O(N)$, $\Omega(1)$ and Θ is undefined.
- ☐ $O(N)$, $\Omega(1)$ and $\Theta(N)$.

Question 2 2/2

If APPEND-function was linear in the worst case, e.g. $O(N)$, but constant in average, what effect would it have on the efficiency of the whole algorithm?

- ☐ Nothing would change.
- ☐ The efficiency in all cases would be times N .
- ☒ The worst case for the algorithm would be $O(N^2)$ and Θ would be undefined.
- ☐ The worst case for the algorithm would be $O(N^2)$, Θ ja Ω would stay the same.

```
1 FindAPair(Array A, Array B):
2   FOR a_index IN A LOOP
3     FOR b_index IN B LOOP
4       IF A[a_index] == B[b_index] THEN
5         RETURN (a_index, b_index)
6       END IF
7     END LOOP
8   END LOOP
```

Question 3 2/2

Which of the following are valid (as strictly as possible) for FindAPair? Note: you can assume $\text{SIZE}(A) == \text{SIZE}(B)$.

- ☐ $O(N)$
- ☐ $O(N \log N)$
- ☒ $O(N^2)$
- ☒ $\Omega(1)$
- ☐ $\Omega(\log N)$
- ☐ $\Omega(N)$
- ☐ $\Omega(N \log N)$
- ☐ $\Theta(N)$
- ☐ $\Theta(N \log N)$
- ☒ Θ is undefined.

Note: The following 3 algorithms have been presented in the previous exercise. Select the strictest approximation.

```
1 Mean(Vector v):
2   int sum = 0
3   FOR elem IN v :
4     sum = sum + elem
5   END LOOP
6   RETURN sum/SIZE(v)
```

Question 4 2/2

What is the efficiency of Mean()?

- ☐ $\Omega(1)$
- ☐ $\Omega(\log N)$
- ☒ $\Omega(N)$
- ☐ $\Omega(N \log N)$
- ☐ $\Omega(N^2)$

```
1 MeanOfRange(Vector v, int start, int end):
2   int sum = 0
3   FOR elem IN RANGE FROM start TO end :
4     sum = sum + elem
5   END LOOP
6   RETURN sum/(end-start +1)
```

Question 5 2/2

What is the efficiency of MeanOfRange()?

- ☒ $\Omega(1)$
- ☐ $\Omega(\log N)$
- ☐ $\Omega(N)$
- ☐ $\Omega(N \log N)$
- ☐ $\Omega(N^2)$

```

1 FindIfOutlier(Vector v, int max_var):
2   FOR elem IN v:
3     IF max_var < |elem - Mean(v)|:
4       RETURN elem
5   END
6   END

```

Question 6 2/2

What is the efficiency of FindIfOutlier()?

- ☐ $\Omega(1)$
- ☐ $\Omega(\log N)$
- ☒ $\Omega(N)$
- ☐ $\Omega(N \log N)$
- ☐ $\Omega(N^2)$

```

1 RecursiveSum(int N)
2   IF N > 1 THEN
3     RETURN N + RecursiveSum(N - 1)
4   ELSE
5     RETURN 1
6   END IF

```

```

1 RecursiveSumEveryThird(int N)
2   IF N > 1 THEN
3     RETURN N + RecursiveSumEveryThird(N - 3)
4   ELSE
5     RETURN 1
6   END IF

```

Question 7 2/2

What is the asymptotic efficiency of RecursiveSum? (Select the strictest approximation.)

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $O(N)$
- ☐ $O(N \log N)$
- ☐ $O(N^2)$

Question 8 2/2

Which of the following are true?

- ☐ RecursiveSum and RecursiveSumEveryThird -algorithm have same execution times for same N and their asymptotic efficiencies are same.
- ☒ RecursiveSum and RecursiveSumEveryThird -algorithm have different execution times for same N but their asymptotic efficiencies are same.
- ☐ RecursiveSum and RecursiveSumEveryThird -algorithm have same execution times for same N but their asymptotic efficiencies are different.
- ☐ RecursiveSum and RecursiveSumEveryThird -algorithm have different execution times for same N and their asymptotic efficiencies are different.

Course week 3

Standard Template Library

Standard Template Library

Question 1 1/1

For a C++ programmer it is important to know how to use STL, because

- ☐ by using the STL, the programmer can cheat since you don't need to actually program anything.
- ☒ the programmer can save considerable time by using the STL's collection of algorithms whose asymptotic performances are known.
- ☐ when using the STL the programmer need not understand what code he/she is using or writing.

STL containers

STL containers

Question 1 3/3

The STL containers can be divided into two groups; sequences and associative containers. The main features of sequences are

- ☐ the sequence elements are stored in consecutive locations in memory
- ☒ the user can place the elements in any order he/she wants
- ☒ the user must specify the position where an element is added or removed
- ☐ the size of a sequence data structure is always small
- ☐ it is impossible for a user to access two consecutive elements of a sequence
- ☒ the sequence elements can be accessed using an iterator or an index

Question 2 1/1

The other group of containers is associative containers, for which

- ☐ the data structure elements are stored in the order of their keys.
- ☐ the data structure elements are always stored in order.
- ☒ an element of the data structure can be found fast by using its key

STL container performance

STL container performance

Question 1 1/1

When trying to write code that runs efficiently, a programmer should choose containers so that

- ☒ operations that are computed frequently are computed as efficiently as possible
- ☐ all operations are computed as efficiently as possible
- ☐ the amount of data that needs to be stored is as small as possible

STL iterators

STL iterators

Question 1 2/2

Regarding iterators in STL, which of the following are true?

- ☐ They have no use in STL.
- ☒ They can be used to indicate a range of elements in a collection.
- ☒ An iterator is like a bookmark between two consecutive elements.
- ☐ They are a third kind of container.

STL iterator performance and categories

STL iterator performance and categories

Question 1 1/1

The iterator category specifies

- ☐ how efficient each operations they perform is.
- ☐ how complicated or difficult it is to use the iterator.
- ☒ what operations they can do.

STL algorithms

STL algorithms

Question 1 2/2

For what reasons is it better to use algorithms from the STL library to compute some task rather than writing your own?

- ☒ Using STL algorithms makes code more readable and understandable.
- ☐ An STL algorithm always has better runtime efficiency than corresponding code that you write yourself.
- ☒ In an STL algorithm special cases and potetial error situations are handled, which mighth be easily forgotten if you rewrite it yourself.
- ☐ When using an STL algorithm, you do not have to know or understand what you are computing.

STL examples

STL examples

Question 1 1/1

Which of the following is true about the STL algorithms?

- ☒ They are building blocks which one can use in forming one's own algorithm to compute one's own task.
- ☐ There is an STL algorithm available for solving all the computational tasks one will ever encounter.
- ☐ They are useless because of the tasks they handle are too simple.

Lambda-functions

Lambda-functions

Question 1 2/2

A lambda function

- ☒ can be given as a parameter to an STL algorithm
- ☒ is an anonymous function
- ☐ always makes the the code less readable and understable
- ☐ are always written on one line

Lambda functions and capturing variables

Lambda functions and capturing variables

Question 1 1/1

When we say a lambda function captures a variable we mean

- ☐ that it will be impossible for any other function, other than the lambda function, to use a variable having the same name as the captured variable
- ☐ that the lambda function deletes a local variable from the calling function
- ☒ that the lambda function makes a copy of a variable or a reference that it uses inside itself

Tuning STL algorithms and containers

Tuning STL algorithms and containers

Question 1 2/2

Which of the following statements, regarding tuning of STL algorithms, are true?

- ☒ By using tuning, we can expand the use of existing STL algorithms.
- ☐ Tuning algorithms is complicated and difficult.
- ☐ After tuning an STL algorithm the code we have produced is impossible to read and understand.
- ☐ Tuning is implemented by rewriting the STL algorithms.
- ☐ When tuning an STL algorithm we rename the STL algorithm to clarify its use in our code.
- ☒ When tuning an STL algorithm we often add additional parameters and lambda functions or named functions to an STL algorithm call.

Iterators

Iterators

You should select the code that will compute given the task. Unless otherwise specified, the iterator type is `std::vector<int>::iterator`. **To learn these concepts, try to do them in your head instead of a compiler etc.**

Question 1 1/1

We want to move iterator `it` forward by 5 positions.

- ☐ `it + 5;`
- ☐ `for ( int i = 0; i <= 5 ; ++i ) { ++it; }`
- ☒ `it += 5;`

Question 2 1/1

We want an iterator that points to the last item. (In the bookmark analogy, the bookmark would be between the two last books.)

- ☒ `it = vec.end(); --it;`
- ☐ `std::prev(vec.back());`
- ☐ `vec.end();`

Question 3 1/1

We want to compute the size of the vector.

- ☐ `for ( auto it = vec.begin(); it < vec.end() ; --it ) { ++koko; }`
- ☒ `vec.end() - vec.begin();`
- ☐ `*vec.begin();`

Question 4 1/1

What does the following code print?

```
1 vector<int> ints = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2 auto iter = ints.begin();
3 auto another = iter + 5;
4 iter += 3;
5 if ( iter < another ){
6     cout << *iter;
7 } else {
8     cout << *another;
9 }
```

- ☒ 3
- ☐ 5
- ☐ 8

Question 5 1/1

What does the following code print?

```
1 vector<int> ints = {0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
2 auto iter = ints.begin();
3 iter = iter * 3;
4 cout << *iter;
```

- ☐ 3
- ☐ 9
- ☒ Nothing, as it doesn't compile.

STL containers

STL containers

Question 1 1/1

Elements are **added** to the end of the data structure, and they are **read & removed** in reverse order, hence the last element added is the first that will be read and removed.

- ☐ array
- ☒ vector
- ☐ forward_list
- ☒ list
- ☐ (multi)set
- ☐ unordered_(multi)set
- ☒ stack
- ☐ queue
- ☒ deque
- ☐ map
- ☐ unordered_map

Question 2 1/1

Elements are **added** to the data structure, and they are **read & removed** in the same order, hence the first added element is the first that will be read and removed.

- ☐ array
- ☐ vector
- ☒ forward_list
- ☒ list
- ☐ (multi)set
- ☐ unordered_(multi)set
- ☐ stack
- ☒ queue
- ☒ deque
- ☐ map
- ☐ unordered_map

Question 3 1/1

Elements are often **added** to the data structure. Often it is necessary to **go through** all the elements in the order in which they were added. Occasionally it is necessary to **go through** all the elements in increasing order, after which the largest element is **removed**.

- ☐ array
- ☒ vector
- ☒ forward_list
- ☒ list
- ☐ (multi)set
- ☐ unordered_(multi)set
- ☐ stack
- ☐ queue
- ☒ deque
- ☐ map
- ☐ unordered_map

Question 4 1/1

Elements are often **added** to the data structure. Often there is a need to **go through** all the elements in increasing order, after which the smallest element is **removed**.

- ☐ array
- ☐ vector
- ☐ forward_list
- ☐ list
- ☒ (multi)set
- ☐ unordered_(multi)set
- ☐ stack
- ☐ queue
- ☐ deque
- ☐ map
- ☐ unordered_map

Question 5 1/1

Elements are often **added** to the end of the data structure. Seldomly an element is **added** in the middle in some designated location. Often there is a need to **get** the i^{th} element (where i varies).

- ☐ array
- ☒ vector
- ☐ forward_list
- ☐ list
- ☐ (multi)set
- ☐ unordered_(multi)set
- ☐ stack
- ☐ queue
- ☒ deque
- ☐ map
- ☐ unordered_map

Question 6 1/1

Elements are often **added** to the data structure. Often it is necessary to **find out**, if an element of given value is in the data structure, and then **remove** the element, if it is found.

- ☐ array
- ☐ vector
- ☐ forward_list
- ☐ list
- ☐ (multi)set
- ☒ unordered_(multi)set
- ☐ stack
- ☐ queue
- ☐ deque
- ☐ map
- ☐ unordered_map

Question 7 1/1

Elements are often **added** to the data structure. Often it is necessary to **find out**, if an element of given value is in the data structure, and then **remove** the element, if it is found. Less often it is necessary to **go through** all the elements in increasing order.

- ☐ array
- ☐ vector
- ☐ forward_list
- ☐ list
- ☒ (multi)set
- ☒ unordered_(multi)set
- ☐ stack
- ☐ queue
- ☐ deque
- ☐ map
- ☐ unordered_map

STL container efficiency

STL container efficiency

Question 1 1/1

N items are inserted into a container.

What would be the asymptotic efficiency of the whole insertion, if the container was a list?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $O(N)$
- ☐ $O(N \log N)$

Question 2 1/1

Consider again the previous question. What would be the asymptotic efficiency if the container was a set?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☐ $O(N)$
- ☒ $O(N \log N)$

Question 3 1/1

A container has N items in arbitrary order. A check is made to see if a certain value is in the container.

What would be the average efficiency of computing the check, if the container is vector?

- ☐ $O(1)$
- ☐ $O(\log N)$
- ☒ $O(N)$
- ☐ $O(N \log N)$

Question 4 1/1

Consider again the previous question. What would be the average efficiency if the container was an unordered_set?

- ☒ $\Theta(1)$
- ☐ $\Theta(\log N)$
- ☐ $\Theta(N)$
- ☐ $\Theta(N \log N)$

Course week 4

Graphs and trees

Graphs and trees

Question 1 1/1

Which of the following is true?

- ☐ A tree may have many root vertices (nodes), a lot of non-leaf vertices but mostly leaf-vertices.
- ☐ In a tree each root has a simple path to its corresponding vertex (node).
- ☒ In a rooted tree there is one root vertex (node) and all the other vertices are its children or grandchildren (of some generation).

Tree data structures and tree traversals

Tree data structures and tree traversals

Question 1 1/1

When you implement a tree using C++, the simplest implementation would be a struct per each node, in which you would store...

- ☒ all data of the node and pointers to its child-nodes.
- ☐ all data of the node, and pointers to both parent-nodes and to all its child-nodes.
- ☐ a pointer to the root node, the data and pointers to all its child-nodes.

Iterating trees

Iterating trees

Question 1 1/1

How do different tree iteration methods differ?

- ☐ They traverse different nodes.
- ☐ Different iteration methods have different efficiencies when you traverse all nodes.
- ☒ They traverse the nodes in a different order.

Invalidation

Invalidation

Question 1 1/1

What does invalidation cause in STL?

- ☐ Inserting and removing elements become asymptotically slower than anticipated
- ☒ Indirect references to some elements (pointers, iterators) become unusable
- ☐ All elements of the container are removed

Amortized performance

Amortized performance

Question 1 1/1

Amortized efficiency means

- ☐ the smallest possible runtime efficiency a sequence of operations can ever achieve
- ☐ the runtime efficiency one hopes to obtain from a sequence of operations
- ☒ the average runtime efficiency for a sequence of operations

Asymptotic performance

Asymptotic performance

Question 1 2 / 2

With regard to verifying an algorithm's asymptotic efficiency, which of the following are true?

- ☐ An algorithm's asymptotic runtime efficiency can be verified by measuring it's actual execution for one input data.
- ☒ An algorithm's asymptotic runtime efficiency can be verified by measuring it's actual execution time for sets of input data whose sizes vary.
- ☒ For one specific input data, the actual execution time of an algorithm will vary because of the other operations the computer is doing.
- ☐ The only way to test an algorithm's runtime efficiency is by measuring the actual execution time of the algorithm.

Increasing the actual performance

Increasing the actual performance

Question 1 3 / 3

Choose the methods that can be used to improve the actual performance of some program.

- ☐ If some algorithm is coded as a single function, divide it into several functions.
- ☒ For a particular computational task, choose an algorithm that computes only what is necessary.
- ☐ Suppose one can either use an STL algorithm or write one's own algorithm. One should always use the STL algorithm.
- ☒ For a particular computational task which can be done using two different algorithms, choose the more efficient.
- ☒ Instead of computing the same quantity numerous times, store the quantity in a variable and recompute it only when it changes.
- ☐ Suppose one can either use an STL algorithm or write one's own algorithm. One should always write one's own algorithm.

Randomization

Randomization

Question 1 1 / 1

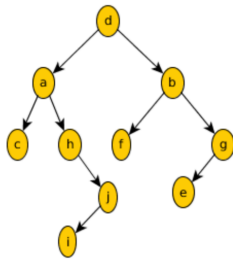
What is a valid reason for using randomization in some procedure?

- ☐ to test the functionality of the algorithm using random data
- ☐ to improve the asymptotic runtime efficiencies of the worst cases of the procedure
- ☒ to try to decrease the probability that a worst case occurs when using the procedure

Trees

Trees

Below is a tree. Answer the following based on that tree. Note: Write answers uncapitalized, in alphabetic order, and use comma to separate items.



Trees

Question 1 2/2

What's the root of this tree?

d

Question 2 2/2

What are the leaves of this tree?

c, e, f, i

Question 3 2/2

What's the height of this tree?

4

Question 4 2/2

How many subtrees can be found from the tree? (Also count the tree itself)

10

Pre-order traversal

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week4/content/?hl=en>

Undo
Reset
Model Answer
Grade

Ohjeet: Reproduce the behavior of binary tree preorder traversal. Click nodes to indicate the order in which the traversal algorithm would visit them.

```

1. preorder(node)
2.   if node == null then return
3.   visit(node)
4.   preorder(node.left)
5.   preorder(node.right)

```

```

graph TD
    12((12)) --> 10((10))
    12 --> 46((46))
    10 --> 22((22))
    10 --> 39((39))
    22 --> 21((21))
    22 --> 39
    46 --> 52((52))
    46 --> 88((88))
    52 --> 51((51))
    52 --> 88

```

In-order traversal

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week4/content/?hl=en>

Undo
Reset
Model Answer
Grade

Ohjeet: Reproduce the behavior of binary tree inorder traversal. Click nodes to indicate the order in which the traversal algorithm would visit them.

```

1. inorder(node)
2.   if node == null then return
3.   inorder(node.left)
4.   visit(node)
5.   inorder(node.right)

```

```

graph TD
    52((52)) --> 35((35))
    52 --> 56((56))
    35 --> 17((17))
    35 --> 47((47))
    17 --> 13((13))
    17 --> 32((32))
    47 --> 49((49))
    47 --> 49
    56 --> 99((99))
    56 --> 99

```

Post-order traversal

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week4/content/?hl=en>

UndoResetModel AnswerGrade

Ohjeet: Reproduce the behavior of binary tree postorder traversal. Click nodes to indicate the order in which the traversal algorithm would visit them.

1. postorder(node)

2. if node == null then return

3. postorder(node.left)

4. postorder(node.right)

5. visit(node)

```
graph TD; 48((48)) --- 44((44)); 48 --- 87((87)); 44 --- 21((21)); 21 --- 26((26)); 87 --- 57((57)); 57 --- 68((68)); 87 --- 97((97)); 97 --- 88((88));
```

Invalidation

Invalidation

```
1 using Iter = std::vector<int>::iterator;
2 std::vector<int> vec = {1, 4, 10, 12, 23, 6, 12};
3
4 for ( Iter it = vec.begin(); it != vec.end(); ++it){
5     if ( *it % 2 == 0 ){
6         vec.insert(it, *it + 1);
7     } else {
8         vec.erase(it);
9     }
10 }
11 Iter biggest = std::max_element(vec.begin(), vec.end());
12 Iter div = std::partition(vec.begin(), vec.end(), [](int i){ return i % 2 == 0;});
13 std::sort(vec.begin(), div);
14 std::sort(div, vec.end());
15 vec.push_back( vec.back() + 2);
16
17 for ( auto i : vec ){
18     std::cout << i << " ";
19 }
```

Question 1 3/3

On what rows in the program above can iterator invalidation happen?

- ☐ 2
- ☐ 4
- ☐ 5
- ☒ 6
- ☒ 8
- ☐ 11
- ☐ 12
- ☐ 13
- ☐ 14
- ☒ 15
- ☐ 17
- ☐ 18

Improved functions and explanations

Improving code efficiency

[main.cpp](#) [download](#) [okCode.pro](#) [download](#)

You should download files main.cpp and okCode.pro. The file main.cpp contains several functions, whose efficiency can be improved. You should make changes to the functions to improve their efficiency. You should submit your modified code, where your changes have the following format: - The old code that you do not need is commented out. - Your new code that replaces the old code and a comment what the efficiency problem was and how your new code fixes it.

You can use the okCode.pro file to build the project in QtCreator. Main-function contains only example calls, and you may change it's contents freely to try out the the functions.

Here is a (silly) example of an 'improved' function:

```
std::vector<int> intsFromZeroTo(int N){
    // std::vector<int> ints; (Old code)
    // The new code.
    std::superFastVector<int> ints;
    // Reason: A superFastVector just sounds faster than an ordinary vector.
    for ( int i = 0; i < N ; i++){
        // ints.push_back(i); (Old code)
        // The new code.
        ints.push_bellybutton(k);
        // Reason: It is much easier to push your belly button than to push your back.
        // Hence, this new code will run faster.
    }
    return ints;
}
```

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
#include <map>

//----- So
me utilities
using ite

r = std::vector<int>::iterator;
using RNG = std::default_random_engine;
void print(std::vector<int> vec){
    std::for_each(vec.begin(), vec.end(), [](auto a){ std::cout << a << " ";});
    std::cout << std::endl;
}

//----- Actual functions

std::vector<int> intsFromZeroTo(int N){
    std::vector<int> ints;

    ints.reserve(N);    //-- New code
    // Reason: Pre-allocate memory avoids repeated reallocations and copying.
    // This improves actual runtime performance.

    for ( int i = 0; i < N ; i++){
        ints.push_back(i);
    }
    return ints;
}

// int min(std::vector<int> vec){                //-- Old code
int min(const std::vector<int>& vec){            //-- New code
// Reason: Passing by const reference avoids copying.

    if ( vec.empty() ){ return 0;}

    // if ( std::is_sorted(vec.begin(), vec.end()) ){
    //     return vec[0];
    // } else {
    //     std::sort(vec.begin(), vec.end());
    //     return vec[0];
    // }
    // }                //-- Old code
    return *std::min_element(vec.begin(), vec.end());    //-- New code
    // Reason: std::min_element finds the minimum in O(n) time
    // without sorting the entire vector.
}

// std::map<int, int> cumulative_sums(std::vector<int> v) {    //-- Old code
std::vector<int> cumulative_sums(const std::vector<int>& v){    //-- New code
// Reason: Passing by const reference avoids copying the input vector.

    // std::map<int,int> sums;    //-- Old code
    std::vector<int> sums;        //-- New code

```

```

    // Reason: Using element values as map keys causes overwriting when duplicates occur,
    // since std::map requires unique keys.
    // Although using the index as the key would fix correctness,
    // prefix sums are naturally indexed by position.
    // Therefore, std::vector is a more appropriate and efficient data structure.

    sums.reserve(v.size());    //-- New code
    // Reason: reserve() reduces reallocations.

    /*
    for (unsigned int i=0; i<v.size(); ++i) {
        if (sums.empty())
            { sums.at(v[i]) = v[i]; }
        else
            { sums.at(v[i]) = sums.at(v[i-1]) + v[i]; }
    }
    */    //-- Old code

    int running = 0;            //-- New code
    for(size_t i = 0; i < v.size(); ++i){    //-- New code
        running += v[i];        //-- New code
        sums.push_back(running);    //-- New code
    }
    // Reason: Recursion is unnecessary because prefix sums are computed sequentially.
    // A loop avoids function call and stack overhead.

    return sums;
}

// void randomized_3part_quicksort(iter begin, iter end, RNG rng)    //-- Old code
void randomized_3part_quicksort(iter begin, iter end, RNG& rng)    //-- New code
// Reason: Passing by const reference avoids copying at every recursion.
{
    if(begin == end) return;    //-- Old code
    if(end - begin <= 1) return;    // New code
    // Reason: A range with 0 or 1 element is already sorted, so we return early
    // to avoid unnecessary pivot selection and partitioning.

    // std::shuffle(begin, end, rng);    //-- 0
    // auto pivot = *(begin + (end - begin)/2);    //-- 0
    std::uniform_int_distribution<std::ptrdiff_t> dist(0, (end - begin) - 1);    //-- N
    int pivot = *(begin + dist(rng));    //-- N

    // Reason: Random pivot selection provides randomization
    // without full shuffling at every recursion.

    iter middle1 = std::partition(begin, end,
                                   [pivot](int val){ return val < pivot; });
    iter middle2 = std::partition(middle1, end,
                                   [pivot](int val){ return !(pivot < val); });
    std::cout << &begin << " " << middle1 - begin << " " << middle2 - begin << " " <<

```

```

end - begin << std::endl;
    randomized_3part_quicksort(begin, middle1, rng);
    randomized_3part_quicksort(middle2, end, rng);
}

int main()
{
    auto vec = intsFromZeroTo(10);
    print(vec);
    auto rng = std::default_random_engine {};
    std::shuffle(vec.begin(), vec.end(), rng);
    std::cout << min(vec) << std::endl;
    print(vec);
    randomized_3part_quicksort(vec.begin(), vec.end(), rng);
    print(vec);
}

```

Course week 5

Heap

Heap

Question 1 1/1

What would be the main benefit of using heap?

- ☐ Keep all items in ascending order in an efficient way
- ☐ Present items in a recursive data structure
- ☒ Keep efficiently record of the largest (or smallest) item

Heap operations

Heap operations

Question 1 1/1

In the heapify algorithm

- ☐ The entire heap is re-ordered
- ☒ The number (key) at the root is moved down, until a correct location is found
- ☐ The locations of the nodes on the same level are switched, until leaves are put into ascending order

Efficiency of heap operations

Efficiency of heap operations

Question 1 1/1

Regarding the runtime efficiencies of the heap algorithms, which of the following is true?

- ☐ Buildheap, Heapify, Heap-Insert and Heap-Extract-Max all have the same asymptotic efficiencies.
- ☐ Of the Buildheap, Heapify and Heap-Insert algorithms, none have a runtime efficiency that is linear in the number of nodes in the heap.
- ☒ The runtime efficiencies of Heapify, Heap-Insert and Heap-Extract-Max are linear in the height of the heap.

Heap as array

Heap as array

Question 1 1/1

When a heap is implemented as an array, which of the following is true?

- ☐ The table can be easily sorted in ascending order.
- ☒ It saves memory and helps in finding the rightmost leaf.
- ☐ the operations are asymptotically faster.

Heap sort

Heap sort

Question 1 2/2

Of the following, what are the advantages of Heapsort?

- ☒ No extra storage space is needed to sort the elements in the heap.
- ☐ It is more efficient than Insertionsort when the number of elements is small.
- ☒ It has the same asymptotic efficiency as other fast sorting algorithms such as Mergesort.
- ☐ You can get both the smallest and the largest item in a constant time.

Graphs

Graphs

Question 1 1/1

A characteristic feature of the graph is that...

- ☐ Elements cannot be removed from it.
- ☐ It also always contains the order of magnitude between the elements.
- ☒ It describes the relationships between elements.

Breadth first search, BFS

Breadth first search, BFS

Question 1 1 / 1

Given a directed graph, in which of the following situations would a breadth first search (BFS) be useful?

- ☒ When one wants to find the path of smallest length from one particular node to each other node.
- ☐ When one wants to find the path of longest length from one particular node to any other node in the graph.
- ☐ When one wants to find the shortest cycles in a graph.
- ☐ When one wants to find the path of smallest length from each node to one particular node.

Depth first search, DFS

Depth first search, DFS

Question 1 1 / 1

Given a directed graph, in which of the following situations would a depth first search (DFS) be useful?

- ☐ When one wants to find all the paths between two given nodes.
- ☒ When one wants to find out, whether there are cycles in the directed graph or not.
- ☐ When one wants to find the longest path between two given nodes.

Implementing graphs

Implementing graphs

Question 1 1 / 1

When implementing a graph in C++, which of the following is true?

- ☐ One should always use the available `std::graph-structure` container.
- ☒ For a given node, somehow one needs to store its adjacent nodes.
- ☐ One should always use a `std::unordered_map` container.

Heap insert

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>

Undo
Reset
Model Answer
Grade

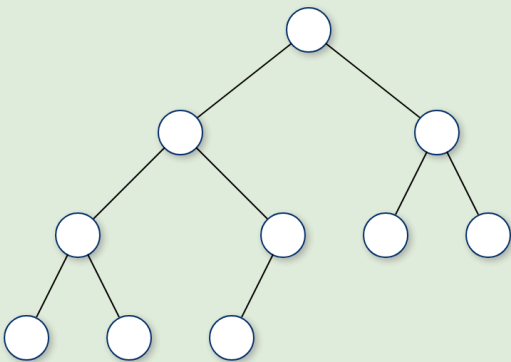
Ohjeet: Insert the stream of keys shown in the upper-left corner into an originally empty **min** heap. The heap is displayed in both its logical tree and physical array forms. Clicking on an element in one will also update the other. You can insert a key by clicking an empty tree node or array cell. Once you insert a new key, be sure to update the key to restore the min heap property as necessary. You can swap two records in the heap by first clicking one, then the other.

1. Algorithm MinHeap-Insert(A, key)
2. heap-size[A] $\leftarrow$ heap-size[A] + 1
3. i $\leftarrow$ heap-size[A] - 1
4. A[i] $\leftarrow$ key
5. while i > 0 and A[Parent(i)] > key
6. do A[i] $\leftarrow$ A[Parent(i)]
7. i $\leftarrow$ Parent(i)
8. A[i] $\leftarrow$ key



Insert values

0	1	2	3	4	5	6	7	8	9



Heap remove

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>



Ohjeet: Perform DeleteMin three times. After each deletion, restore the **min** heap property. Click the "Decrement heap size" button to remove the last position from the heap. You can swap records in the heap by first clicking one key and then another.

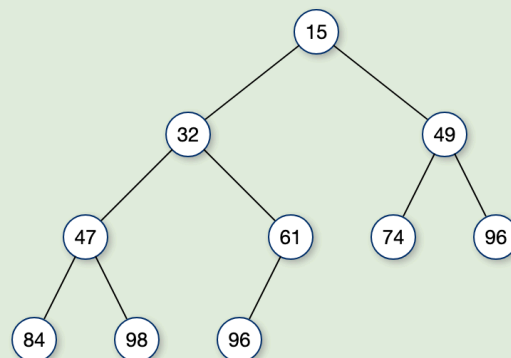
```

1. Algorithm 1 Heap-DeleteMin(A)
2.   min ← A[0]
3.   Swap(A[0], A[heap-size[A]-1])
4.   heap-size[A] ← heap-size[A] - 1
5.   Min-Heapify(A, 0)
6.   return min
7.
8. Algorithm 2 Min-Heapify(A, i)
9.   l ← Left-child-index(i)
10.  r ← Right-child-index(i)
11.  if l < heap-size[A] and A[l] < A[i] then
12.    smallest ← l
13.  else
14.    smallest ← i
15.  end if
16.  if r < heap-size[A] and A[r] < A[smallest] then
17.    smallest ← r
18.  end if
19.  if smallest ≠ i then
20.    Swap(A[i], A[smallest])
21.    Min-Heapify(A, smallest)
22.  end if

```

Decrement heapsize

15	32	49	47	61	74	96	84	98	96
0	1	2	3	4	5	6	7	8	9



Heap build

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>

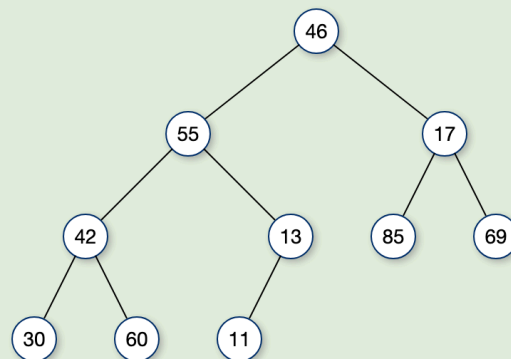
Ohjeet: Your task is to reproduce the behavior of the buildheap algorithm to create a **min** heap from the data initially in the array. The heap is displayed in both its logical tree and physical array forms. Clicking on an element in one will also update the other. You can swap two records in the heap by first clicking one, then the other.

```

1. Algorithm 1 Build-Min-Heap(A)
2.   for i ← heap-size[A]/2 - 1 downto 0 do
3.     Min-Heapify (A, i)
4.   end for
5.
6. Algorithm 2 Min-Heapify(A, i)
7.   l ← Left-child-index(i)
8.   r ← Right-child-index(i)
9.   if l < heap-size[A] and A[l] < A[i] then
10.    smallest ← l
11.  else
12.    smallest ← i
13.  end if
14.  if r < heap-size[A] and A[r] < A[smallest] then
15.    smallest ← r
16.  end if
17.  if smallest ≠ i then
18.    Swap(A[i], A[smallest])
19.    Min-Heapify(A, smallest)
20.  end if

```

46	55	17	42	13	85	69	30	60	11
0	1	2	3	4	5	6	7	8	9



Heapsort

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>

Ohjeet: Reproduce the behavior of heapsort for the **maximum** heap below. You can swap keys by clicking the first one and then the second one in either of the representations (array or binary tree). Begin by swapping the last key with the largest key, and reducing the size of the heap by one (by clicking the "Decrement heap size" button). After that, restore the heap property again.

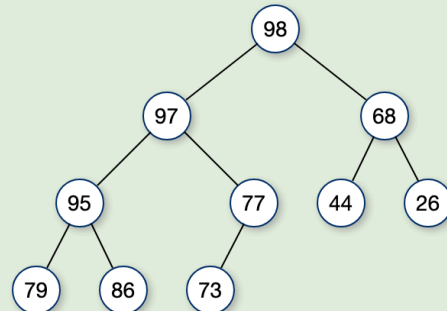
```

1. Algorithm 1 Heapsort(A)
2.   while heap-size[A] > 0 do
3.     Swap(A[0], A[heap-size[A] - 1])
4.     heap-size[A] ← heap-size[A] - 1
5.     Max-Heapify(A, 0)
6.
7. Algorithm 2 Max-Heapify(A, i)
8.   l ← Left-child-index(i)
9.   r ← Right-child-index(i)
10.  if l < heap-size[A] and A[l] > A[i] then
11.    largest ← l
12.  else
13.    largest ← i
14.  end if
15.  if r < heap-size[A] and A[r] > A[largest] then
16.    largest ← r
17.  end if
18.  if largest ≠ i then
19.    Swap(A[i], A[largest])
20.    Max-Heapify(A, largest)
21.  end if

```

Decrement heap size

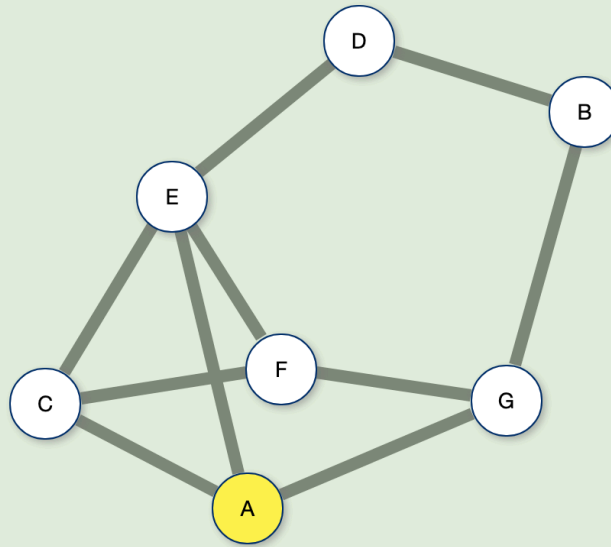
98	97	68	95	77	44	26	79	86	73
0	1	2	3	4	5	6	7	8	9



Breadth First Search

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>

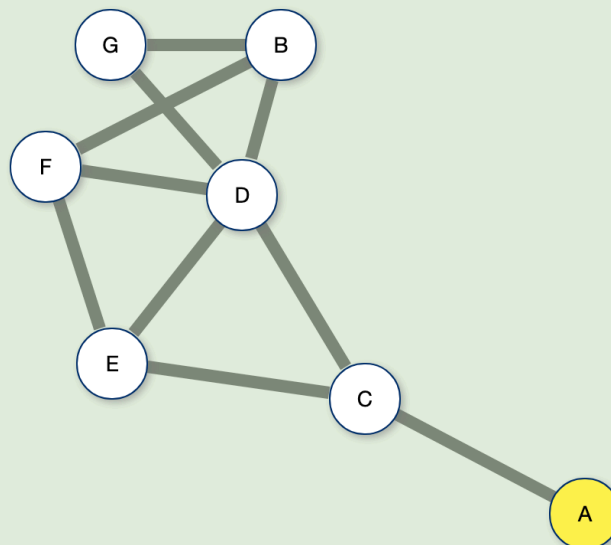
Ohjeet: Reproduce the behavior of the BFS algorithm for the graph below. Just click on the **edges** in the order that they will be traversed by the BFS algorithm. Start with Node A. If there is more than one node that could be visited next, choose the one that comes first in alphabetical order.



Depth First Search

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week5/content/?hl=en>

Ohjeet: Reproduce the behavior of the DFS algorithm for the graph below. Just click on the **edges** in the order that they will be traversed by the DFS algorithm. Start with Node A. If there is more than one node that could be visited next, choose the one that comes first in alphabetical order.



Exercise Maze

From each intersection in a maze, one could go left, right, up and/or down. Answer the questions: 1. How would you represent a such maze as a graph datastructure (maze has walls so you cannot move to every direction from every point)? 2. Write a C++ struct, that could be used as a node in a graph that represents a maze.

1. A maze can be represented as a graph by treating each walkable cell or intersection as a vertex (node). If it is possible to move directly from one cell to another (up, down, left, or right) without hitting a wall, then an edge is created between those two vertices. The graph can be stored using an adjacency list, where each node keeps a list of its neighboring nodes that can be reached directly.

2.

```
#include <vector>
```

```
struct MazeNode
```

```
{
```

```
    int x;
```

```
    int y;
```

```
    std::vector<MazeNode*> neighbors;
```

```
};
```

Course week 6

Weighted graphs

Weighted graphs

Question 1 1/1

The difference between a weighted graph and an ordinary graph is that in a weighted graph

- ☐ each node has a weight, and the weight determines whether edges can enter the node or not.
- ☒ each edge has a weight associated with it and the weight is some number which can be used to find the cheapest route.
- ☐ no cycles are permitted.

Dijkstra's algorithm

Dijkstra's algorithm

Question 1 1/1

Dijkstra's algorithm uses a procedure called RELAX. What does RELAX do?

- ☐ For each path from the starting node to some other node currently in the priority queue, RELAX decreases the path length.
- ☐ RELAX removes a node from the priority queue if that node is not reachable from the starting node.
- ☒ RELAX checks whether the length of a new path from the starting node to some other node is less than the length of the current path to the same node.

A*-algorithm

A*-algorithm

Question 1 1/1

The A-star algorithm solves the single pair shortest path problem. With regard to the A-star algorithm, which of the following is true?

- ☒ The A-star algorithm is guaranteed to find a solution if it has a way to compute a lower bound for the distance between two nodes.
- ☐ The A-star algorithm is guaranteed to find a solution only if the input graph has a cycle.
- ☐ The A-star algorithm is guaranteed to find a shorter path than that found by Dijkstra's algorithm.

Efficiency of graph algorithms

Efficiency of graph algorithms

Question 1 1/1

Suppose the same graph is used as input for different graph algorithms. Which of the following is true?

- ☒ BFS and DFS have better efficiencies than Dijkstra (using O-notation).
- ☐ A* has a better efficiency than Dijkstra (using O-notation).
- ☐ BFS has a better efficiency than DFS (using O-notation).

Hash tables

Hash tables

Question 1 1/1

With regard to hash tables, which of the following is true?

- ☐ The elements in a hash table are stored in two separate vectors.
- ☐ The elements in a hash table are stored in decreasing order of the keys.
- ☒ The elements in a hash table are stored in buckets, whose locations are provided by the hash function.

Hash table implementation

Hash table implementation

Question 1 1/1

There are two standard ways of resolving collisions, open hashing and closed hashing. When inserting a new item to the hash table and a collision occurs ...

- ☒ in opening hashing the new item is added to some linked list and in closed hashing the new item is inserted in some empty bucket.
- ☐ in open hashing a new bucket is added to the end of the hash table and in closed hashing some existing item in some bucket is overwritten by the new item.
- ☐ in open hashing the new item is inserted in between two buckets and in closed hashing the new item is stored in a new hash table.

Efficiency of hash table

Efficiency of hash table

Question 1 1/1

For a hash table to have good run time efficiency it is necessary that ...

- ☐ there should be exactly one less bucket than there are items stored in the hash table.
- ☒ the hash function distributes the keys uniformly amongst the buckets.
- ☐ the items to be stored in the hash table are in increasing or decreasing order.

Question 2 1/1

Rehashing is necessary when ...

- ☐ the number of items changes.
- ☒ the load factor becomes larger than some acceptable limit.
- ☐ the number of keys change.

Hash functions

Hash functions

Question 1 4/4

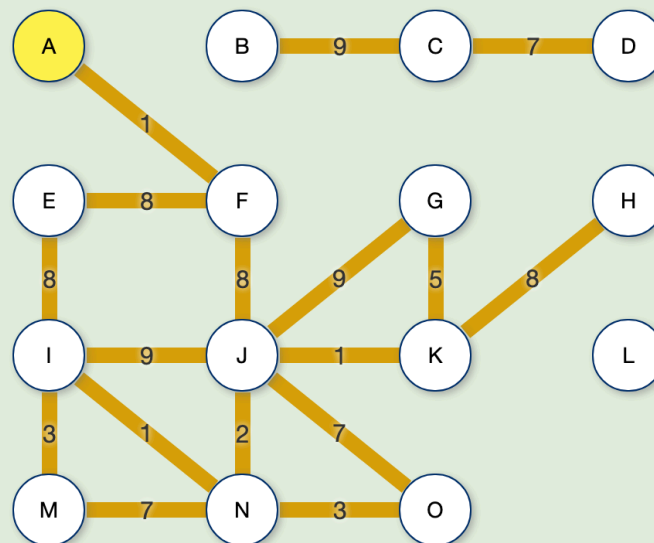
What are the properties of a good hash function?

- ☒ It computes quickly.
- ☐ It randomly maps keys to buckets.
- ☐ It distributes the keys amongst the buckets in accordance with a normal (Gaussian) distribution.
- ☐ It ignores as much of the data in the key as possible.
- ☒ It distributes the keys amongst the buckets uniformly.
- ☒ It uses as much of the data in the key as possible.
- ☒ For a given key, it always yields the same bucket.
- ☐ It always maps two keys of the same length to the same bucket.

Dijkstra's algorithm

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week6/content/?hl=en>

Ohjeet: Reproduce the behavior of Dijkstra's Algorithm for the given graph below. Click on the **edges** in the order they are traversed by the algorithm. Start with Node A. In case of similar costs, add nodes alphabetically.



Exercise Dijkstra1

In the following questions it is assumed we are trying to compute a shortest path from a given node s to a given node t . The pseudocode in procedure DIJKSTRAPAIR presents a modified Dijkstra's algorithm that will solve this problem.

```

1 DIJKSTRAPAIR(s, t, G)
2 /* Starting from source node s finds the shortest path to a target node t given the weighted graph G */
3 forEach node x in G
4   x.color = white, x.d = ∞, x.π = NIL
5 end
6 s.color = gray, s.d = 0
7 INSERT(Q, s, 0)
8 while Q is not empty
9   x = EXTRACT_MIN(Q)
10  /* Check if target node has been found or not. */
11  if x == t then
12    return
13  end
14  forEach node y in x.Adj
15    y.old = y.d, RELAX(x, y)
16    if y.color = white then
17      y.color = gray, y.π = x
18      INSERT(Q, y, y.d)
19    else
20      if y.d < y.old and y.color != black then
21        /* Take into account that y.old < y.d */
22        INSERT(Q, y, y.d)
23      end
24    end
25  end
26  x.color = black
27 end
28 end
29
30 RELAX(x, y)
31 if y.d > x.d + w((x,y)) then
32   y.d = x.d + w((x,y)), y.π = x
33 end

```

Exercise 1

Suppose we wish to add the following condition:

We want the shortest path from s to t that contains node x .

For example, in the above graph we might want the shortest path from F to H that contains node A. Suggest how DIJKSTRAPAIR can be used *without changes* to solve this.

The problem can be solved by running the procedure DIJKSTRAPAIR twice without modifying it.

First, run DIJKSTRAPAIR(s, x, G) to find the shortest path from the source node s to the required intermediate node x . Then, run DIJKSTRAPAIR(x, t, G) to find the shortest path from x to the target node t .

By concatenating these two paths (without repeating node x), we obtain the shortest path from s to t that passes through node x .

Exercise Dijkstra2

Suppose we wish to add the following condition:

We want the shortest path from s to t which does not contain node x .

For example, in the above graph we might want the shortest path from F to H that does not contain node D . What condition do you need to add to DIJKSTRAPAIR to take this condition into account? Also describe where the addition should be.

To ensure that node x is not included in the shortest path, the algorithm must never visit or relax node x .

Therefore, node x should be marked as already processed during initialization, and a condition should be added inside the loop over adjacent nodes so that, if a neighbor is x , it is skipped.

1.

After initialization part:

(line 3) forEach node x in G

(line 4) $x.color = white, x.d = \infty, x.\pi = NIL$

(line 5) end

Add:

(line 6) $forbidden_node.color = black$ /* The forbidden node must never be used */

2.

In the neighbor iteration loop beginning:

(line 15) forEach node y in $x.Adj$

Add:

if $y == forbidden_node$ then

 continue /* Skip the forbidden node */

end

Open hashing

<https://plus.tuni.fi/COMP.CS.300/spring-2026/week6/content/?hl=en>

Ohjeet: In this exercise your task is to perform the given operations on the hash table. The operations are in the stack, and each item in the stack has a key value and label which tells the type of the operation.

When looking for a key in the linked list, remember to **mark all nodes before clicking on the index where the operation will actually take place**. The hash function can be seen below.

$$h(k) = k \bmod 11$$

Undo Reset Model Answer Grade

$$300 \bmod 11 = 3$$

Operation stack



Hash Table

0	1	2	3	4	5	6	7	8	9	10

Hash tables

Hash tables

```
1 int y(x, buckets) {
2   int result = 0;
3   while ( x != 0 ){
4     // Multiply the first and last digits of x and add to the result
5     result += last_digit(x) * first_digit(x)
6     x = remove_first_and_last_digit(x)
7   }
8   return (result mod buckets)
9 }
```

Question 1 4 / 4

What bucket will the key 12345678 hash into, when the hash function is the y shown above and there are 100 buckets?

60

Question 2 8 / 8

Which one of the following hash functions distributes the following items {"Eetu", "Kai", "Nelli", "Kaisa", "Elias"} most evenly, if there are 9 buckets? (Note. All of the hash functions given are actually quite bad.)

- ☐ the first letter's running number in the alphabet mod 9.
- ☐ the length of the key mod 9.
- ☒ the first letter's running number in the alphabet multiplied by the length of the key mod 9.

Question 3 4 / 4

What is the load factor of the hash table from the previous question? Give your answer as a fraction.

5/9

Course week 7

Binary search trees

Binary search trees

Question 1 1/1

Which of the following properties does a binary search tree (BST) have?

- ☐ Each key stored in a BST is either a zero or a one.
- ☐ Binary search tree is used to store the search results of other structures.
- ☒ For each node x , all keys smaller than the key of x are found in x 's left subtree and all keys greater than x 's key are found in x 's right subtree.

Question 2 1/1

With regards to the smallest and the largest keys in a binary search tree (BST), which of the following is true?

- ☒ The smallest key is found at the leftmost node in the BST and the largest is found at the rightmost node.
- ☐ As there is no simple way to find either the smallest or the largest, it is always necessary to test all of the keys to find one or the other.
- ☐ The largest key is always found at the root and the smallest is found at the deepest leaf.

Balanced binary search trees

Balanced binary search trees

Question 1 1/1

Why unbalanced binary tree is so inefficient?

- ☒ In the worst case the BST consists of one long branch and the runtime efficiency of a search is linear in n .
- ☐ In the worst case even though the BST is balanced, the runtime efficiency of a search is still linear in n .
- ☐ There is no order at all in the nodes of the BST and the runtime efficiency of a search is $n \log(n)$.

Question 2 1/1

With regard to performing a rotation in a binary search tree (BST) at node x , which of the following is true?

- ☐ After the rotation the BST property is reversed, so x 's left subtree contains keys larger than x 's key and x 's right subtree contains keys that are smaller than x 's.
- ☒ In the rotation, one of x 's children becomes x 's parent and the BST property is preserved.
- ☐ In the rotation, the leaves that are the farthest from x become the nodes that are closest to x and the nodes that are the closest to x become leaves that are the farthest from x .

Different binary search trees

Different binary search trees

Question 1 1/1

How are rotations used in an AVL tree?

- ☒ Rotations are performed when the difference of the heights of the left and right subtrees becomes too large.
- ☐ Rotations are only used at leaves and thereby the balance of the AVL tree is maintained at depths where the leaves are.
- ☐ In every addition

Question 2 1/1

Which of the following is *not* a property of a red-black binary search tree?

- ☐ A red node cannot have a red parent.
- ☐ The root must always be black.
- ☒ The number of red nodes must be at least half the number of black nodes.